Islamic University of Technology

Lab 7: Basic Types

CSE 4108 : Structured Programming Lab

Section 1B

MIA, NFS, IAC, ASH

November 05, 2021

1 Lab Tasks

You do not need array for this lab, and so **DO NOT USE ARRAY**. You can process character by character by taking inputs in buffer from the user using the following code.

```
char ch;
// The following loop will continue until the user enters a newline
while((ch = getchar()) != '\n')
{
    // Do something with ch
}
```

You can find helpful concepts like converting between uppercase and lowercase in the provided document related to this chapter in the theory section.

1.1 Have the Pi and Eat it too

Early methods to calculate the value of pi (π) were geometric. However the real accuracy came with the use of infinite series techniques. Write a program that uses one of the earliest infinite series methods to compute the value of π as a floating-point number.

Taking the first 4 elements of the series, the value of pi obtained is 2.8952381.

$$\frac{\pi}{4} = \frac{1}{1} - \frac{1}{3} + \frac{1}{5} - \frac{1}{7}$$

Taking the first 10 elements of the series, the value of pi obtained is 3.0418396.

$$\frac{\pi}{4} = \frac{1}{1} - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \frac{1}{13} - \frac{1}{15} + \frac{1}{17} - \frac{1}{19}$$

[Sample Program]
Input 1

```
Enter number of elements: 4
```

Output 1

```
Pi: 2.8952381
```

Note that the values of π got progressively closer to the true value. For greater accuracy, your program should use variables of type `double` rather than `float`. Have the program produce output to a precision of 7 decimal places.

1.2 Consonant Struggle of Life

Write a program that counts the number of consonants (all letters from *a-z* except *a*, *e*, *i*, *o*, and *u*) in a sentence.

[Sample Program]
Input 1

```
Enter a sentence: Hello World.
```

Output 1

```
Your sentence contains 7 consonant(s).
```

[Hint] *Be careful, there are both upper case and lower case consonants.*

1.3 Camel vs Snake

Write a program in C that would take a variable name as input in snake case and convert it to camel case.

In snake case, there are hyphens (-) instead of spaces and all the letters are in lowercase. In camel case, the name starts with a lowercase letter and the first letter of every subsequent word is capitalised while all other letters are in lowercase.

[Sample Program]

Input

```
Enter a snake case: my_variable_that_i_have_named_with_few_words
```

Output

```
Camel Case:  myVariableThatIHaveNamedWithFewWords
```

[Hint] *You can detect when a word starts by keeping track of the previous character in a separate variable. Try to write the upper to lower and lower to upper logic by yourself.*

1.4 Save the Date

Write a program that asks the user for a date in the *DD Month* format, then displays the date in *DD-MM* format. To avoid months starting with the same letters, the dates will only be between 01 January and 30 April. Character needs to be used in this task.

[Sample Program]

Input 1

```
Enter a DD Month date: 9 March
```

Output 1

```
Equivalent DD-MM date: 09-03
```

Input 2

```
Enter a DD Month date: 10 February
```

Output 2

```
Equivalent DD-MM date: 10-02
```

[Hint] *Considering only the first letter of the month i.e,* `J` *for* `January` *is enough.*

[Hint] *You can detect when a word starts by keeping track of the previous character in a separate variable. Since the first letter encountered is always the first letter of first word, you can initialize that separate variable as SPACE. Also, be careful about the last word in the sentence.*

1.5 Up Down - Down Up

Write a C program that takes a sentence as input and *reverses* its case. By reversing, it means an upper case letter would be lower case and a lower case letter would be upper case. All the punctuation marks will remain as it is.

[Sample Program]

Input

```
Enter a sentence: What? No.
```

Output

```
wHAT? nO.
```

[Hint] *You can detect when a word by detecting spaces as like the previous problem. Try to write the upper to lower and lower to upper logic by yourself.*

1.6 L33T

Leetspeak is a language most commonly used by aspiring hackers to show their elite status. It works through replacement of characters with their similar glyphs. You want to propose a modified leetspeak to show that you are an elite member of the community. Your modified leetspeak replaces characters with only numbers of similar glyph. So, you've come up with a set of character replacement rules as following:

```
0=O, 1=L, 2=Z, 3=E, 4=A, 5=S, 6=G, 7=T, 8=B, 9=J
```

According to your rules, the word `LEET` will become `1337`. Now, you'll write a C program to automate this conversion process. The program will take a sentence as input and convert it into your modified leetspeak. The letters for which there's no replacement will remain as it is. There will be no punctuation mark and all letters will be in upper case.

[Sample Program]

Input 1

```
Enter sentence: WE ARE LEET HACKERZ
```

Output 1

```
W3 4R3 1337 H4CK3R2
```

[Hint]